

32-post copy and script bank

LC-STRATEGY-V4.0.0 · Selected strategy. Complete proposed copy; exact assets await approval.
Sunil examples are illustrative until product-story evidence is cleared.

Roadmap and launch gates

Audience: Experienced engineer or technical lead · Practical question

Caption

The roadmap has a launch date. Does it have a reason to believe the system will be ready?

Imagine reviewing an agent that prepares a daily plan. The demo works. The team can explain the architecture. The calendar says it is time for a pilot.

Then you ask what happens when the inputs are incomplete. Or when the tool returns something the system cannot use. The answers are still being worked out.

That is the conversation I would want before agreeing to the date.

A date helps people coordinate. A release gate explains what must be true: which behaviour has been tested, what evidence is available, who owns the decision and what happens if the condition is missed.

I would put both on the roadmap. I would also name what we are deliberately leaving out. Otherwise, an extra week can become an extra feature instead of time to resolve the original uncertainty.

For this illustrative system, the next useful milestone might be a bounded trial with a review owner, rather than a wider release.

Look at one milestone on your own roadmap. Could someone outside the team explain what evidence would close it?

Explain audience and programme outcome

Audience: Experienced engineer or technical lead · Explore programme

Caption

You can understand the components of an agent and still have difficult questions about its design.

What is it allowed to do? How will you evaluate it? Which failure should stop a release? How would you explain the trade-off to your team?

At The Living Craft, I bring those questions into practical work. Members build a working agentic system, explain the architecture behind it and revise their work through continuous feedback.

The open cohort is for people with prior system-design exposure who want to strengthen how they design, critique and guide agentic systems.

Explore the programme and bring a learning question to the application.

learning.thelivingcraft.ai

Image — Design agentic systems. Guide your team.

Build a working system. Explain your decisions. Develop your reasoning through continuous feedback.
30 live hours plus independent work.
Explore the open cohort.

Whether an agent is needed

Audience: Experienced engineer or technical lead · Practical question

Caption

Before choosing an agent framework, I would ask what actually needs an agent.

This illustrative planning example starts with a task that sounds like one job. Looking closely reveals several different jobs—and some are better handled by ordinary code.

Try the same separation on your next proposal: where is judgment needed, and where would a rule be enough?

Spoken script

Imagine your team is building a service that prepares a daily plan. Someone suggests an agent, and the conversation immediately turns to models and frameworks.

I would pause one step earlier. What decisions are we asking the system to make?

Checking whether a required field is present does not need a model. Neither does enforcing a fixed limit. Those are jobs we can describe and test directly.

Choosing between several reasonable options, with incomplete context, is a different problem. That is where I would examine whether a model adds useful judgment—and whether an agent needs tools to gather information or act.

I would also ask what happens when the answer is wrong. Can someone review it? Can we recover? How will we tell whether the result was useful?

In this illustrative example, the design might end up with a small agentic part surrounded by quite ordinary software. That is a reasonable outcome.

The question is whether the approach earns its complexity for this task.

Take one proposed agent in your team. Separate the fixed rules, the uncertain decisions and the permitted actions. Which part still needs an agent after you have done that?

Demonstrate a use-case review

Audience: Experienced engineer or technical lead · Try the review

Caption

A useful design review can make an agent smaller.

Consider an illustrative expense assistant. It reads a request, checks policy and prepares a recommendation. A separate service authorises any payment.

I would review each step before deciding where a model belongs. Fixed limits can be enforced in code. Unclear descriptions may need interpretation. Missing evidence needs a defined next step.

The result is a clearer allocation of responsibility, with fewer decisions left to an open-ended prompt.

Try the review questions in this example against one workflow you know.

Panel 1 — Start with the job

An expense assistant should help resolve a request. “Use an agent” does not explain the job.

Panel 2 — Separate the fixed rules

A spending limit can be checked in code. A model does not need to reinterpret it on every run.

Panel 3 — Find the uncertain decision

An unclear description may need interpretation or another question. Name what information would resolve it.

Panel 4 — Bound the action

Preparing a recommendation and authorising a payment carry different responsibilities. Define each permission.

Panel 5 — Decide how to check it

Use cases with known outcomes, review ambiguous cases and test what happens when evidence is missing.

Panel 6 — Review your own workflow

Which steps need interpretation? Which need tools? Which can stay deterministic?
Illustrative teaching example.

Model selection

Audience: Experienced engineer or technical lead · Practical question

Caption

“Which model should we use?” can hide several different decisions.

Imagine a system with a planner, a checker and a writer. The planner chooses a path. The checker examines constraints. The writer explains the result.

Giving them all the same model is convenient. It may also mean paying for capability a step does not need—or asking a model to do a check that code could perform more reliably.

I would compare the jobs before comparing the models. Then I would test candidate approaches on the work each role actually does.

A second model can offer another view, but a different vendor or model family does not by itself prove independent judgment.

For one role in your system, write down what would make you change your current choice.

Panel 1 — Choose for the job

A planner, checker and writer can have different needs. Start by naming the decision each one owns.

Panel 2 — Define acceptable behaviour

What must this role get right? Which failure would make the output unusable?

Panel 3 — Compare on your cases

Use representative inputs and the same acceptance criteria. Include difficult and incomplete cases.

Panel 4 — Count the surrounding cost

Include retries, review time and failed work alongside model charges. A cheaper call can cost more overall.

Panel 5 — Test the checker too

A second model can share mistakes. Compare its judgments with known cases and qualified human review.

Panel 6 — Make the choice revisitable

Record the reason, the limitation and the evidence that would trigger a new comparison.
Illustrative model-selection review.

Demonstrate a reasoned comparison

Audience: Experienced engineer or technical lead · Try the review

Caption

Suppose a cheaper model produces a fluent answer but repeatedly misses a required condition.

Would you choose it?

I would want to know whether that condition is essential, whether another part of the system enforces it, and what the missed check costs someone downstream.

This illustrative comparison is the kind of reasoning I want members to practise: make a choice, explain the evidence and stay open to revising it.

Use the questions here when a model comparison turns into an argument about a leaderboard.

Panel 1 — Compare a decision

Illustrative case: choose a model for drafting a service plan that must respect fixed constraints.

Panel 2 — Start with the constraint

One candidate writes clearly but misses a required condition. Decide whether that failure is acceptable before looking at price.

Panel 3 — Inspect the whole system

Can a deterministic check detect the omission? What happens after it is detected? Include the cost of that recovery.

Panel 4 — Compare like with like

Use the same representative cases and criteria. Record latency, successful work, failures and review effort.

Panel 5 — Explain the trade-off

“It is cheaper” leaves out the consequence. Explain what improves, what becomes harder and what remains uncertain.

Panel 6 — Say what would change your mind

A decision is easier to review when the next piece of evidence is explicit. What would you test next?

Architecture

Audience: Experienced engineer or technical lead · Apply

Caption

The architecture diagram looks complete. Can it explain a failed run?

In this illustrative example, I follow a planner, a checker and the part of the system that decides whether to retry. The boundaries become easier to discuss when we put an actual behaviour beside the boxes.

That connection between a working system and its design is central to The Living Craft.

Explore the open cohort: learning.thelivingcraft.ai

Spoken script

Imagine reviewing an agent architecture with a planner, some tools and a checker. The arrows form a neat loop.

Now suppose the checker rejects the result. What happens next?

If the answer is “the agent tries again,” I would ask what changes on that next attempt. Is there new information? Another valid option? Or will the system repeat the same failing step?

In this illustrative design, I would give the checker a structured result: which condition failed, what evidence it used and whether the workflow can recover.

Then I would make the retry decision explicit. The orchestrator needs a stopping rule and a path for cases it cannot resolve. A model should not have to invent those responsibilities while it is running.

Finally, I would ask for a trace that lets someone reconstruct the decision without exposing unnecessary personal data.

Now the diagram can explain a behaviour. We can build the loop, test a failure and revise the design when the evidence calls for it.

At The Living Craft, that is the connection I want members to practise: a working agentic system, the reasoning behind it and feedback that helps improve the next decision.

If you already work with system design, explore the open cohort and bring a question you would like to work through.

Show a design-review conversation

Audience: Experienced engineer or technical lead · Explore programme

Caption

What does feedback on an architecture actually sound like?

Here is an illustrative review. An agent keeps retrying after its checker rejects the output. The diagram contains all the expected components, but the next action is unclear.

I would start by asking what changes between attempts. That question can reveal a missing recovery path more quickly than adding another component to the diagram.

At The Living Craft, the working system gives feedback something concrete to refer to. Members explain their choices, examine the behaviour and revise the design.

Explore the programme: learning.thelivingcraft.ai

Panel 1 — A review starts with behaviour

Illustrative example: the checker rejects an output and the agent tries again. The same rejection repeats.

Panel 2 — Ask the next question

What new evidence or valid option does the next attempt have? If nothing changes, why would it succeed?

Panel 3 — Locate the decision

The checker identifies the failure. The workflow needs an explicit decision about recovery, stopping or human review.

Panel 4 — Make a change you can test

Bound the attempts. Define the unresolved outcome. Record enough evidence to explain why the workflow stopped.

Panel 5 — Run the difficult case again

Does it stop correctly? Can a reviewer understand the failure? What uncertainty remains?

Panel 6 — Practise the reasoning

Build, explain, examine and revise through continuous feedback.
Design agentic systems. Guide your team.
learning.thelivingcraft.ai

Cost control

Audience: Experienced engineer or technical lead · Practical question

Caption

A cheap model call is not the same thing as a cheap completed task.

Imagine an agent that keeps asking for another attempt because the available options do not satisfy the request. Each attempt is inexpensive. Together, they produce a bill and no useful result.

I would look at the loop before shopping for a cheaper model.

What work repeats unnecessarily? Could a shared calculation be done ahead of time? What must stay specific to this request? When does the system stop trying?

Moving work off the request path can help, but it creates another question: how fresh does the prepared result need to be?

And a dashboard alone does not limit a run. A runtime ceiling needs an enforcement point, a defined outcome when reached and a way to investigate what happened.

In this illustrative case, I would track cost per successfully completed task, alongside the failures and review effort that the average could hide.

Choose one expensive workflow. Can you explain its worst permitted run, as well as its average one?

Introduce cost-ceiling worksheet

Audience: Experienced engineer or technical lead · Resource

Caption

A cost limit becomes useful when someone can point to where it is enforced.

The cost-ceiling worksheet turns that conversation into a reviewable table: the unit being limited, the reason for the limit, the enforcement point, the stop behaviour and the evidence from testing it.

I would use it before a pilot, then revisit it when the task, traffic or model changes. It does not supply a universal budget. Your use case has to justify the number.

Use the worksheet to examine one workflow, including what the user experiences when a limit is reached.

Panel 1 — What bounds a run?

Use the cost-ceiling worksheet to connect each limit to a reason, an owner and an enforcement point.

Panel 2 — Count the work

Model calls are one part. Include retries, tool calls, retrieval, background work and human review.

Panel 3 — Choose the unit

Per run, per person and per period answer different questions. Record which one each ceiling controls.

Panel 4 — Define the stop behaviour

What is saved? What does the user see? Who handles unresolved work? A limit without a recovery path is incomplete.

Panel 5 — Test the boundary

Exercise repeated requests and concurrent runs. Check that enforcement behaves as designed and leaves useful evidence.

Panel 6 — Review one workflow

Complete the cost-ceiling worksheet, then identify the next test your team needs.

Learning and memory

Audience: Experienced engineer or technical lead · Practical question

Caption

“The user preferred this yesterday” is a useful observation. It is not a permanent instruction.

Imagine a planning assistant that treats one choice as a lasting preference. It becomes wonderfully consistent at making the wrong suggestion.

I would separate what happened from what the system inferred. The event is evidence. The belief needs a scope, a way to become outdated and a way for the person to correct it.

Confidence can help decide whether to ask a question, but a confidence value is not permission to take a consequential action.

Look at one stored preference in your system. Can the person find it, correct it and understand where it applies?

Panel 1 — An observation is not a person

Illustrative example: someone chooses an option once. The system treats it as a preference everywhere.

Panel 2 — Keep the evidence separate

Store the observed event separately from the inferred belief so the reasoning can be examined.

Panel 3 — Give the belief a scope

When and where does it apply? A workday choice may tell you little about a weekend.

Panel 4 — Make uncertainty useful

Ask a relevant question when evidence is weak. Avoid pretending a numerical confidence is a verified probability.

Panel 5 — Allow correction and change

People change their minds. Make corrections effective, and revisit beliefs that have become stale.

Panel 6 — Keep permission explicit

Confidence about a preference does not authorise a payment, message or other consequential action.

Explain building feedback and revision

Audience: Experienced engineer or technical lead · Explore programme

Caption

A design decision can look sensible until you see what the system does with it.

That is why I want members to build alongside the discussion. The working system gives a review something observable: an assumption, a trace, a result or an unresolved case.

Continuous feedback helps identify what is working, what needs attention and what to try next. The next version gives us something new to examine.

This is the learning process at The Living Craft. If you already have system-design exposure, explore how the programme fits your next responsibility.

learning.thelivingcraft.ai

Image — Build. Examine. Revise.

Make a design decision. See what the system does. Use continuous feedback to identify the next useful change.

Explore The Living Craft.

Incorrect output

Audience: Experienced engineer or technical lead · Practical question

Caption

The checker can be right while the system is stuck.

Consider an illustrative planning agent. A candidate breaks a required constraint, so the checker rejects it. The next attempt selects the same unsuitable candidate. The checker rejects it again.

Nothing unsafe reaches the user. Nothing useful reaches them either.

I would examine the available candidates and the recovery path, as well as the checker. Is there another valid option? Can the workflow ask for missing information? Does it know when to stop?

This is why I am careful with claims that a single extra model will solve incorrect output. A check has a scope. It may detect one class of failure and miss another. It also needs a useful consequence when it finds a problem.

The right mix can include deterministic checks, model-based review and human judgment, depending on what we need to assess.

Take one failure your system catches. Can you explain what happens after it is caught?

Review a failure and its limitations

Audience: Experienced engineer or technical lead · Try the review

Caption

“The bad output was blocked” is the start of a review, not the end.

In this illustrative case, an agent has no valid candidate left after a constraint check. Repeating the same selection cannot resolve that.

I would review prevention, detection and recovery separately. Each answers a different question. The team should also name the judgments its checks cannot make.

Use this example to inspect one blocked workflow. Does the system know how to finish safely, even when it cannot finish successfully?

Panel 1 — The checker caught it

Illustrative case: a selected item violates a required condition. The checker rejects it.

Panel 2 — Now inspect the next step

Does another valid option exist? Repeating a failed choice can leave the workflow stuck.

Panel 3 — Prevent what you can

Validate candidates before selection where the requirements are known and testable.

Panel 4 — Match checks to failures

Code can enforce explicit constraints. Model and human reviews may help with judgments those checks cannot express.

Panel 5 — Name what remains uncertain

A constraint check does not establish that the result is appropriate in every context. State its limits.

Panel 6 — Design the unresolved outcome

Stop, request information or route for review. Make that outcome understandable to the person waiting.

Evaluation

Audience: Experienced engineer or technical lead · Apply

Caption

“The tests passed” is difficult to review when nobody can say which decisions the tests support.

For an illustrative agent pilot, I would separate requirements that block release from concerns that need a named review decision. Then I would record the cases, results and gaps behind each judgment.

A passing suite tells us about the cases we tested. It does not prove that every future situation is covered.

At The Living Craft, members connect this evidence to a working system and practise explaining the design choices it supports.

Explore the open cohort: learning.thelivingcraft.ai

Panel 1 — What does passing mean?

Start with the behaviour the system must demonstrate, and the decision the evidence is meant to support.

Panel 2 — Separate the release decisions

Some failures block release. Other trade-offs need an explicit owner and a documented decision.

Panel 3 — Use suitable reviewers

Choose code, model-based or human grading for the property being examined. Check the reviewers too.

Panel 4 — Make coverage visible

Record representative cases, difficult cases and known gaps. Passing tested cases is not universal assurance.

Panel 5 — Keep the evidence connected

Link the result to the system version, configuration and inputs so the decision can be revisited.

Panel 6 — Practise evidence-led review

Build a working agentic system, explain the evidence and revise through feedback.
learning.thelivingcraft.ai

Introduce evaluation-gates worksheet

Audience: Experienced engineer or technical lead · Resource

Caption

Before a release meeting, I would want to know which failures can stop the launch—and who can accept the remaining trade-offs.

The evaluation-gates worksheet makes those decisions explicit. It connects a requirement to test cases, a reviewer, a result and the action that follows.

It also leaves room for uncertainty. A result without coverage information can sound more reassuring than it deserves.

Use the worksheet against one release decision. If a result changes tomorrow, would the team know what to do?

Panel 1 — A result needs a consequence

Use the evaluation-gates worksheet to connect evidence to a release decision.

Panel 2 — Define the requirement

Describe the behaviour being checked. Avoid a broad label such as “safe” without explaining what it means here.

Panel 3 — Record the cases

Include ordinary, difficult and incomplete inputs. State the coverage and the gaps.

Panel 4 — Choose the reviewer

Use deterministic checks, model-based grading or qualified human review according to the requirement.

Panel 5 — Name the decision owner

State what blocks release, what needs review and where the decision will be recorded.

Panel 6 — Make the next action clear

Complete the worksheet for one gate. What changes if it fails? What remains unknown if it passes?

User isolation

Audience: Experienced engineer or technical lead · Practical question

Caption

A shared table can make a system simpler. It can also make a missing filter much more consequential.

Imagine an assistant that can use shared reference material and a person's private records. A query returns both. Everything works until another endpoint forgets which records belong to whom.

I would want the access boundary to survive that mistake, rather than depend on every caller remembering the same condition.

The storage pattern is only part of the design. We need to know how identity reaches the data layer, where permissions are enforced, and how the same rules apply to retrieval, background jobs, caches and logs.

I would test a request using another person's record identifier. I would also test what changes when someone leaves an organisation or withdraws access.

This is an illustrative example, not a claim that one database pattern fits every application.

Pick one private record in your system. Can you follow its access rules through every path that can return it?

Explain prerequisites and learning goals

Audience: Experienced engineer or technical lead · Apply

Caption

You do not need a particular job title to have a useful system-design conversation.

I would rather understand a decision you have owned. What was the system meant to do? Which constraint shaped it? What did you choose, and what did you give up?

The Living Craft open cohort expects prior system-design exposure and readiness to build and revise through feedback. It is not an introduction to programming.

You can work with an industry case. You do not need to bring confidential employer material.

Describe your experience and learning goal when you apply: learning.thelivingcraft.ai

Image — Start with your responsibility

What have you designed, reviewed or helped a team decide?

Bring system-design exposure, a learning goal and readiness to build.

Apply to The Living Craft.

Deployment

Audience: Experienced engineer or technical lead · Practical question

Caption

A small team can spend weeks designing a deployment pipeline and still have no clear answer to “what prevents this change reaching production?”

For an illustrative service, I would start with the failures we need to prevent. Then I would choose controls that address them and record what we are deferring.

A fast check before merge, separate production access and a deliberate promotion decision may address different risks. None substitutes for the others.

The useful plan explains the order, the owner and the next test—not just the tools.

Which deferred deployment control in your team has a clear trigger for reconsideration?

Panel 1 — Start with the failure

Illustrative case: a change passes review but its scheduled job is missing after deployment.

Panel 2 — Check before merge

Run relevant fast checks. State what they can detect and which checks require a different environment.

Panel 3 — Separate production access

Make it difficult for routine development or review work to modify production data. Test the boundary.

Panel 4 — Make promotion deliberate

Identify who approves the release, what evidence they review and which version moves forward.

Panel 5 — Verify after deployment

Check health and scheduled work. Define rollback or recovery before the release needs it.

Panel 6 — Record the deferrals

Name what is not implemented, why, and what change in risk or scale would trigger it.

Introduce deployment checklist

Audience: Experienced engineer or technical lead · Resource

Caption

A deployment checklist is useful when it identifies an owner and produces evidence.

“Check production” is difficult to act on. “Verify the scheduled job ran, link the result and record who reviewed it” gives the team a clear task.

The deployment checklist covers change checks, access boundaries, promotion, post-release verification and recovery. It also asks what has been deferred and when that decision should be revisited.

Use it for one release. Adapt the controls to the system's risks; completing a checklist alone does not establish production readiness.

Panel 1 — Can someone operate this release?

Use the deployment checklist to make each action, owner and evidence visible.

Panel 2 — Before the change

Identify the version, the checks required and any data migration or compatibility concern.

Panel 3 — At the access boundary

Verify that development and review activity cannot accidentally use production permissions.

Panel 4 — At promotion

Record the approver, evidence and version. Know what must happen before traffic or jobs use it.

Panel 5 — After release

Check the service and scheduled work. Confirm alerts, recovery and the person who will respond.

Panel 6 — Keep the unfinished work visible

Record deferred controls with a reason and a revisit trigger. Use the checklist on your next release.

Meaningful metrics

Audience: Experienced engineer or technical lead · Practical question

Caption

An app open can mean “this helped me.” It can also mean “I am checking why this went wrong.”

In this illustrative example, I look at what a planning assistant's metrics can—and cannot—tell us.

Choose one metric on your dashboard. What decision would change if it moved?

Spoken script

Imagine a planning assistant whose usage dashboard looks healthy. People keep opening the app.

That could be good news. It could also mean they keep returning to correct the plan.

I would want the dashboard to help distinguish those possibilities.

Suppose someone rejects a recommendation and explains why. That event tells us something more specific than an app open. It still needs interpretation. Was the recommendation unsuitable, the context outdated, or the explanation unclear?

And the people who take time to correct a system may not represent everyone who uses it. Silence is not proof of satisfaction.

In this illustrative example, I would connect activity with outcomes, explicit feedback and selected conversations with users. I would keep the limits of each signal visible.

Then I would ask what the team can do with the result. If a metric changes, does it tell us to inspect a category, revise a rule or ask another question?

A dashboard becomes useful when it helps the team choose its next action.

Take one number you report every week. Write the decision it supports beside it. If that is difficult, examine whether you need another signal or a clearer question.

Explain progress through work and feedback

Audience: Experienced engineer or technical lead · Explore programme

Caption

Finishing an assignment and understanding its design are different things.

At The Living Craft, I want the working system to make the reasoning visible. Why did you draw that boundary? What did you observe? What would you change after the review?

Continuous feedback helps members identify strengths, gaps and follow-up work. A revision can be useful even when it makes the architecture smaller.

The programme focuses on developing capability through practical work. It does not offer a certificate as a substitute for that work.

Explore the learning process: learning.thelivingcraft.ai

Image — Make progress visible

Explain a decision. Examine the result. Use feedback to identify the next change.

A working system gives the review something concrete.

Delivery channels

Audience: Experienced engineer or technical lead · Apply

Caption

The channel decision can change the product before the model does.

Imagine designing an assistant for a conversation involving several people. The whiteboard shows one shared thread. Later, the team discovers that the chosen delivery route does not support the interaction they assumed.

Now the work includes separate messages, consent, reply handling and a way to keep context consistent.

This is an illustrative scenario. The specific constraints depend on the platform, account and current policies. I would verify them before promising the experience.

I would start with the conversation we need to support: who participates, who can initiate it, what each person can see and what happens when delivery fails. Then I would compare the available channels.

At The Living Craft, these surrounding decisions belong in the architecture discussion alongside the agent itself. Members build, explain their choices and revise through feedback.

If you want to strengthen how you approach those decisions, explore the open cohort:
learning.thelivingcraft.ai

Support employer funding decision

Audience: Employer-sponsored individual · Apply

Caption

If you want your employer to support the programme, give your manager a decision they can evaluate.

Explain the responsibility you want to develop, why it matters to your role and how you will make room for the work. Include the confirmed schedule and final fee when they are available.

I would also agree what you can share afterwards without exposing confidential programme or company material. A discussion of a design principle may be more useful than a promise to transform a production system.

Apply as an individual and mention employer support. An organisation buying a programme for a team follows a separate route.

learning.thelivingcraft.ai

Panel 1 — Make the funding decision clear

Explain the capability you want to develop and why it matters to your current responsibilities.

Panel 2 — Name a relevant decision

For example: reviewing an agent architecture, interpreting evaluation evidence or explaining an operating trade-off.

Panel 3 — Show the commitment

The programme includes 30 live hours plus independent work. Confirm the actual schedule and workload before requesting approval.

Panel 4 — Use the final offer

Provide the confirmed fee, tax treatment, payment and access terms. Do not ask a manager to approve an assumption.

Panel 5 — Agree a useful follow-through

Discuss what learning you can share with your team. Do not promise an unscoped production implementation.

Panel 6 — Keep the route simple

Apply as an individual and mention employer funding.

learning.thelivingcraft.ai

Changing models

Audience: Experienced engineer or technical lead · Practical question

Caption

A lower price is a reason to test another model. It is not the result of the test.

This illustrative comparison follows the same task across two candidates and asks what evidence would support a switch.

Before your next model change, decide which result would make you keep the current approach.

Spoken script

Imagine a team considering a cheaper model for an existing workflow. The demonstration looks good, and the price difference is attractive.

I would want a comparison that resembles the work we actually run.

Start with a representative set of inputs, including difficult cases. Reuse the current prompt and decision logic where the comparison allows it. If a candidate needs a different setup, record that difference so we know whether we are comparing models or whole configurations.

Run without writing to production or taking external actions. Check that tools and background effects are disabled too; a flag named dry run is not evidence by itself.

Then compare the outcomes that matter. Did either candidate miss a required condition? How often did it need another attempt or a human review? What happened to latency and the cost of completed work?

Because outputs can vary, one good response does not settle the comparison. Examine repeat runs and the cases where the candidates disagree.

This is an illustrative protocol, not a claim that every model change needs the same experiment.

Before the test, write down what would stop the switch. That makes it easier to keep the decision grounded when the cheaper result looks tempting.

Explain commitment and independent work

Audience: Experienced engineer or technical lead · Ask about cohort

Caption

The time commitment is part of deciding whether this cohort fits your life.

The programme includes 30 live hours with me, plus independent work. The practical work needs room in your schedule as well as the live conversations.

Before joining, confirm the session dates, the amount and timing of independent work, and the arrangements for access or missed sessions. The final offer should make the fees and terms clear too.

If you are unsure how this fits your responsibilities, ask about the cohort before committing.

learning.thelivingcraft.ai

Image — Make room for the work

30 live hours with Sunil, plus independent work.

Confirm the schedule, workload and full terms before joining.

Ask about the cohort.

Silent failures

Audience: Experienced engineer or technical lead · Practical question

Caption

No error was reported. Did the job succeed—or did it never run?

Imagine a service that prepares tomorrow's output overnight. A scheduled run is missed. The next run prepares the following day, leaving the original gap untouched.

The dashboard has no exception to show because the missing work never started.

I would want evidence of expected completion, not only records of failure. Which period should have output? Was it produced? Is it complete? Who is told when it is missing?

Recovery needs similar care. A catch-up job should recognise completed work so it does not create duplicates. It should also know which missing period matters first and whether late output is still useful.

In this illustrative example, monitoring the schedule and designing recovery belong together. An alert can tell a person where to look. It cannot repair a workflow that has no way to fill the gap.

Choose one scheduled job. What would tell you that it had quietly stopped running?

Separate member sponsorship and enterprise

Audience: Member and enterprise buyer; separate routes · Choose enquiry route

Caption

Employer funding can mean two different things.

One person may want support to join the open cohort. An organisation may want a programme scoped for a group. Those need different conversations.

For an individual place, I want to understand the member's experience, learning goal and readiness. For a team programme, the conversation also covers participant experience, the technical sponsor, industry context and an agreed schedule.

Enterprise programmes normally involve 8–10 participants; final scope is agreed separately.

Choose the route that matches the purchase when you enquire: learning.thelivingcraft.ai

Image — One member or a team?

Employer-sponsored individual: apply to the open cohort.

Organisation purchasing group learning: start an enterprise enquiry.

The scope and buying process differ.

Consent and security

Audience: Experienced engineer or technical lead · Practical question

Caption

“Pending” is a state. It should not quietly become permission.

Imagine an assistant that can send a message to another person. The data model distinguishes permission granted, refused and not yet established. The sending code checks only whether it was refused.

The diagram and the behaviour now disagree.

I would test that boundary directly, including withdrawal and queued messages. This illustrative engineering review does not establish compliance with any particular law or platform policy. Those requirements need their own current review.

Pick one permission in your system. What actually happens when it is missing?

Panel 1 — Read the missing state

Illustrative case: a recipient has not granted permission. The system must know what that means for the requested action.

Panel 2 — Separate the states

Granted, refused and not established should have deliberate behaviour. Do not let a default decide silently.

Panel 3 — Check the action boundary

Test the code that sends or acts, including alternate paths. A correct field in the schema is not enough.

Panel 4 — Keep useful evidence

Record the basis, scope and relevant history of permission without collecting unnecessary personal data.

Panel 5 — Test a change of mind

Check withdrawal, queued work and retries. A later action must respect the current permission state.

Panel 6 — Review the actual requirements

Engineering controls support the design. Applicable legal and platform requirements need separate, current review.

Answer application and fit questions

Audience: Experienced engineer or technical lead · Apply or enquire

Caption

An application starts a fit conversation. It does not commit you to a purchase.

Tell me about your system-design experience, the decisions you currently handle and what you want to develop. You can also say whether you are exploring employer funding.

You do not need to send confidential architecture, customer records or source code. An industry example and a clear learning question are enough to begin.

Fit, the final offer, payment and attendance confirmation are separate steps. If you need an answer before applying, use the cohort enquiry route.

learning.thelivingcraft.ai

Image — Bring a question, not confidential files

Describe your experience and what you want to develop.

Apply for a fit conversation, or ask about the cohort first.

learning.thelivingcraft.ai

Specialist models

Audience: Experienced engineer or technical lead · Apply

Caption

The best-known model may not be the best fit for one narrow task.

In this illustrative voice example, I look at the evidence needed to choose a specialist—and why a result in one language cannot stand in for another.

If you want to practise making and explaining decisions like this, explore The Living Craft open cohort.

learning.thelivingcraft.ai

Spoken script

Imagine adding voice to an assistant used in several languages. It is tempting to choose a familiar model and treat speech as another input and output option.

I would start with the conversation people need to have.

Can they correct a recognition error? Does the system handle the words and names they actually use? Can they switch to text when voice is unsuitable?

A specialist may perform better for a particular task or language. That is a reason to compare it, not a reason to assume it wins everywhere.

I would use realistic samples and people qualified to judge the target language. A result in one language does not establish quality in another. I would also compare latency, data handling, cost and the fallback when the service is unavailable.

This is an illustrative review. The interesting decision is how the evidence changes the choice for the experience we need to deliver.

That is the practice at the centre of The Living Craft: build a working agentic system, explain its design and use continuous feedback to improve the reasoning behind it.

If you already have system-design exposure and want to strengthen how you design, critique and guide these systems, explore the open cohort. Bring your experience and a question you want to work on.

Invite application or enquiry

Audience: Experienced engineer or technical lead · Apply or enquire

Caption

If the next stage of your work involves designing agentic systems or helping a team review them, consider whether this is the practice you need.

At The Living Craft, members build a working agentic system, explain their architecture decisions, examine the evidence and revise through continuous feedback.

The programme includes 30 live hours with me, plus independent work. Prior system-design exposure is expected. Confirm the schedule, workload, fees and full terms during the fit and offer process.

Apply with your experience and one learning question. If you need to clarify something first, ask about the cohort.

learning.thelivingcraft.ai

Image — Design agentic systems. Guide your team.

Build a working system. Strengthen how you explain and review its design.
Apply to the open cohort.
learning.thelivingcraft.ai